```
clear all; clc; close all
```

The following line of code loads a previously generated set of fictitious data.

```
addpath('mcmcstat')
addpath('kde')

load mixed_oscillator2  %this is a set of dynamic data

data.ydata = y_mixed(:,1:20)'; %adjust the number of columns to evaluate the
                               %statistical model
data.xdata = t;
data.xdata = data.xdata(:);
% data.ydata = data.ydata(:);

%define initial guess for parameters
z0 = 1.5;
k = 1e5;
C = 30;
m = 1;

theta = [z0
         k
         C
        ];

%model parameter range
params = {
    {'z_0', theta(1), 0, inf}
    {'k', theta(2),0,inf}
    {'C', theta(3),0,inf}
    };
```

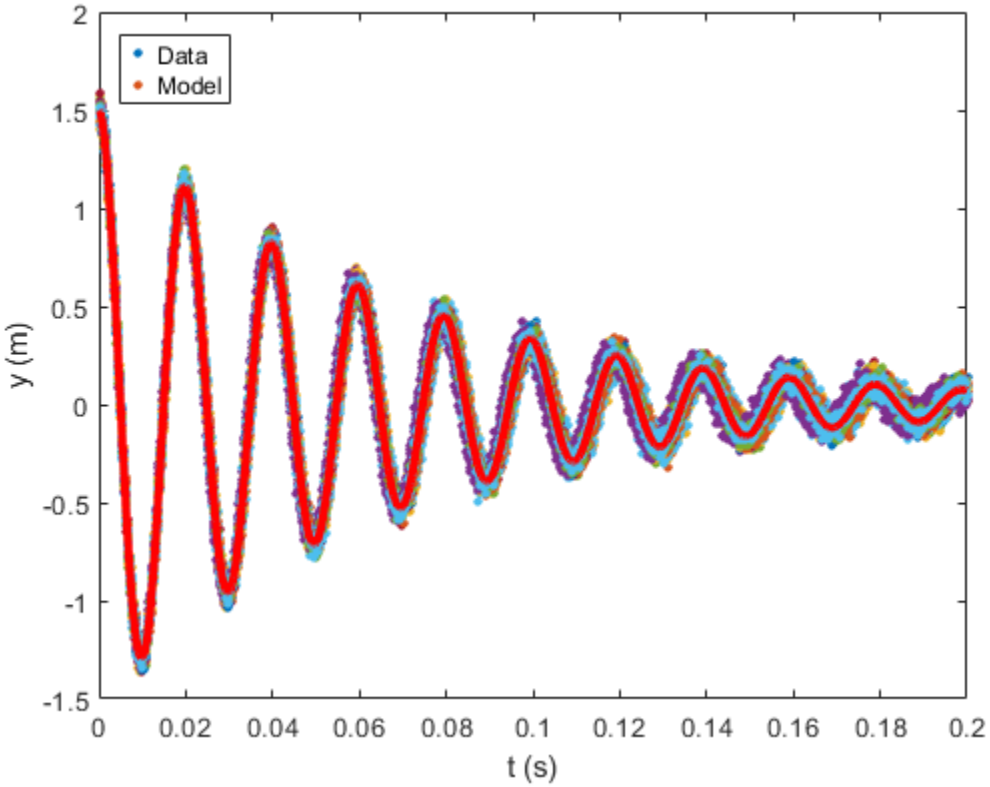
Call function that computes the error between the model and data

```
ssfun = @SS_func;
model.ssfun=ssfun;
```

The next lines of code check the initial parameter guesses prior to running Bayesian parameter calibration to see of the values given reasonable results with respect to the data.

```
[y_model] = mass_spring_model_Bayesian(theta,data.xdata);

figure(1)
plot(data.xdata,data.ydata(:,:),'x','MarkerSize',3,'Linewidth',2)
hold on
plot(data.xdata,y_model,'r-','Linewidth',3)
hold off
xlabel('t (s)')
ylabel('y (m)')
legend('Data','Model','Location','NorthWest')
```



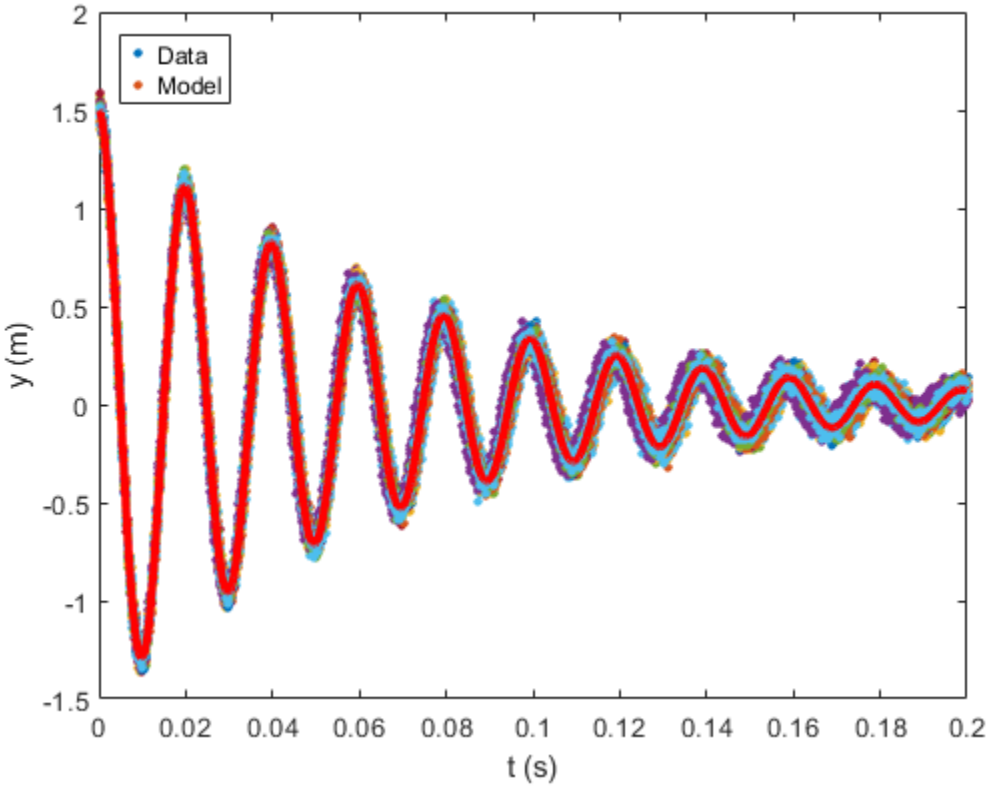
The Bayesian analysis is calculated here.

```
model.sigma2 = 1e-4;       %initial guess on variance
model.S20 = model.sigma2; %prior for sigma2
model.N0  = 1;             %prior accuracy for S20
options.updatesigma = 1;  %update variance as part of the inference
options.method = 'dram';  %this applies the DRAM algorithm
model.N  = length(data.xdata); %number of data points

options.nsimu = 50000; %number of iterations in the Metropolis method
[results, chain, s2chain]= mcmcrun(model,data,params,options);
chainstats(chain,results) %print chain statistics
```

```
Sampling these parameters:
name    start [min,max] N(mu,s^2)
z_0: 1.5 [0,Inf] N(0,Inf)
k: 100000 [0,Inf] N(0,Inf)
C: 30 [0,Inf] N(0,Inf)
MCMC statistics, nsimu = 50000

                mean          std       MC_err          tau       geweke
----------------------------------------------------------------------------
       z_0      1.506   0.0059473  0.00012415       12.268      0.99877
         k      99996       40.154      0.63301       9.3293      0.99999
         C     30.627      0.18173   0.0018167       10.268      0.99931
----------------------------------------------------------------------------
```

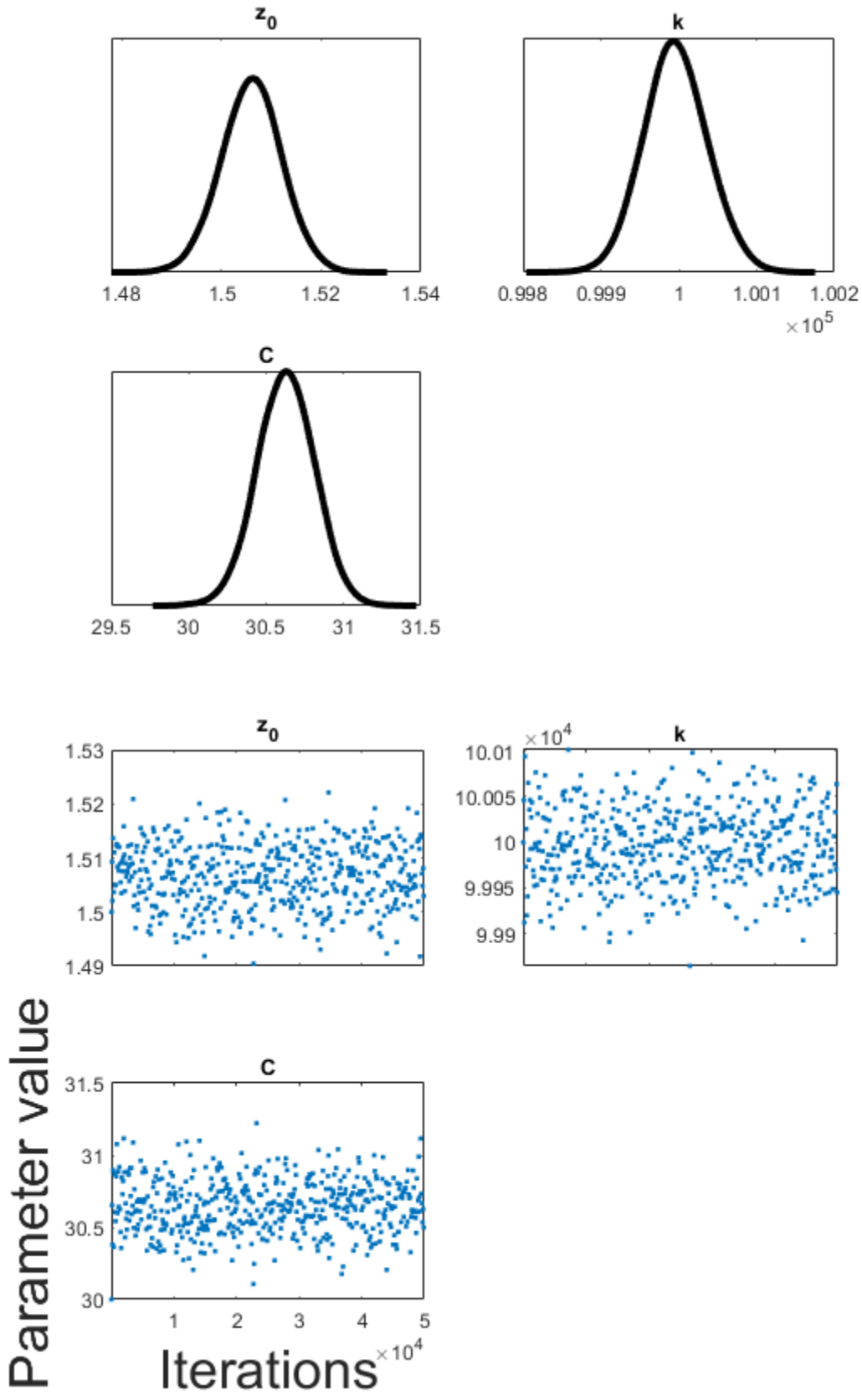


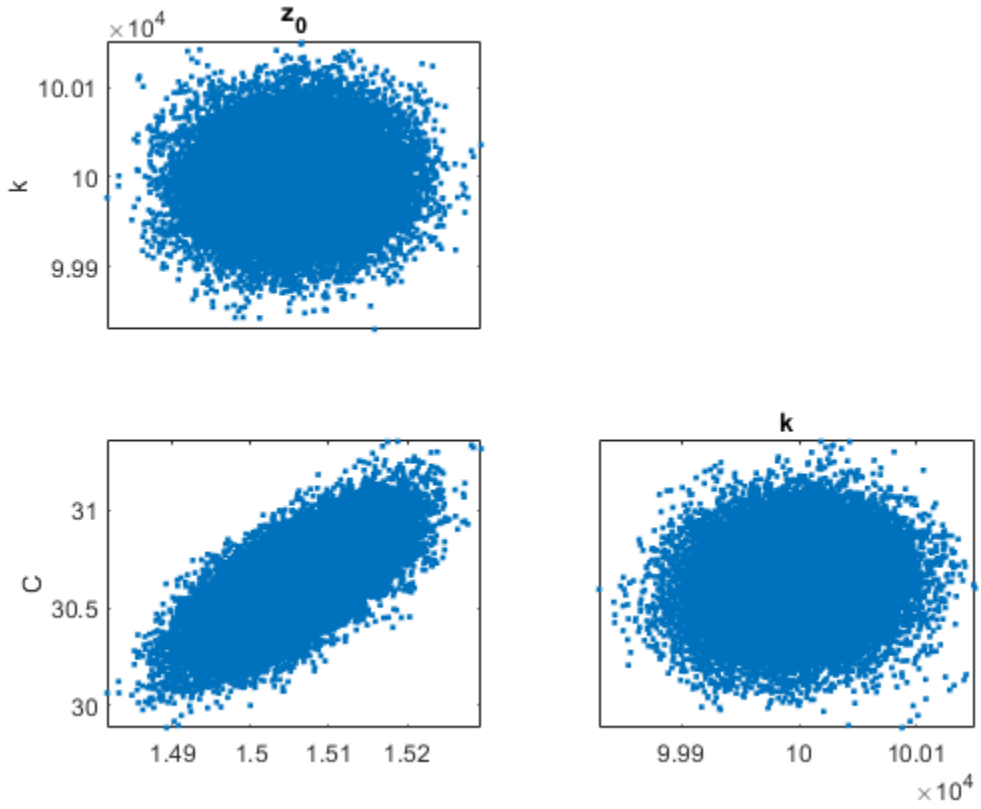
Plot the statistical results. Note that Figure 4 containing pair correlations will not plot except for the nonlinear case where there is more than one parameter.

```
figure(2)
mcmcplot(chain(:,:),[],results,'denspanel',2);

figure(3); clf
mcmcplot(chain(:,:),[],results.names,'chainpanel')
xlabel('Iterations','Fontsize',24)
ylabel('Parameter value','Fontsize',24)

figure(4)
mcmcplot(chain,[],results,'pairs');
```

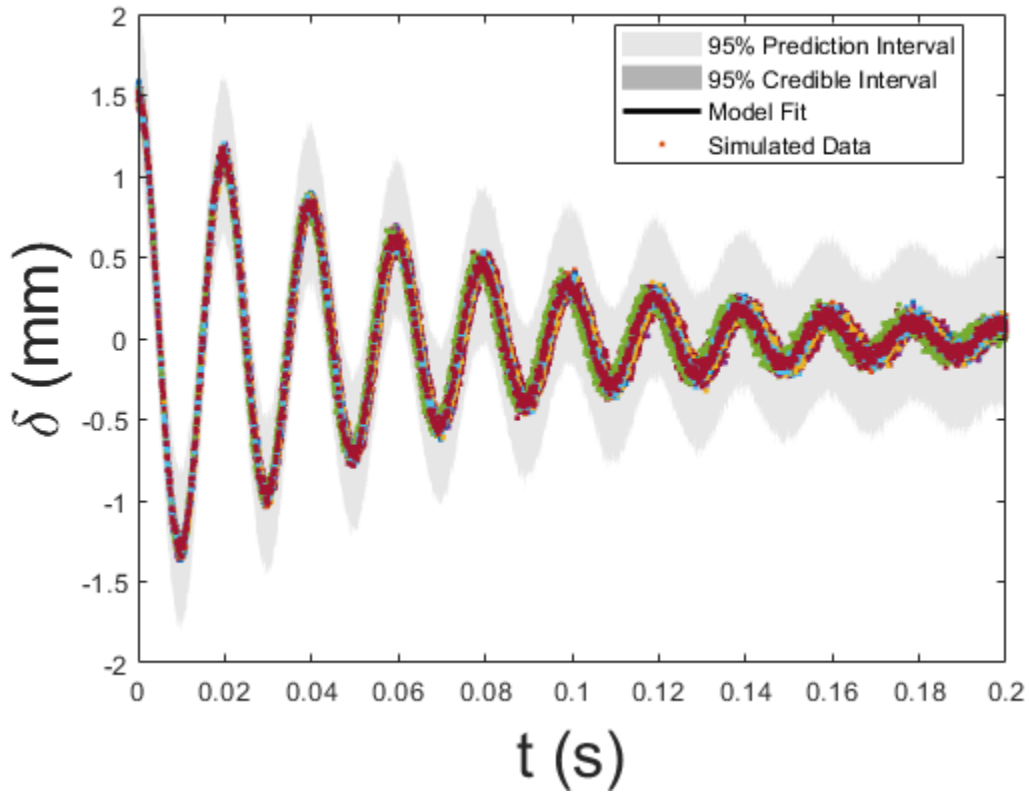

z_0
1.48
1.5
1.52
1.54
k
0.998
0.999
1
1.001
1.002
$\times 10^5$
C
29.5
30
30.5
31
31.5
z_0
1.53
1.52
1.51
1.5
1.49
k
$\times 10^4$
10.01
10.005
10
9.995
9.99
C
31.5
31
30.5
30
1
2
3
4
5
$\times 10^4$
Parameter value
Iterations



Compute the credible and prediction intervals

```
modelfun1 = @(d,th)mass_spring_model_Bayesian(th,d); % NOTE: the order in
which d and th appear is important

nsample = 500; %number of sample iterations of the model used to construct
the interval bounds
               %the default interval bounds are 95% prediction/credible
               %bounds
out = mcmcpred(results,chain,s2chain,data.xdata,modelfun1,nsample);
figure(5)
modelout = mcmcpredplot(out);
hold on
plot(data.xdata,data.ydata,'.','linewidth',1)
hold off
xlabel('t (s)','Fontsize',24);
ylabel('\delta (mm)','Fontsize',24);
legend('95% Prediction Interval','95% Credible Interval','Model
Fit','Simulated Data','Location','Best')
```



Published with MATLAB® R2024b